# Native Isaac ROS on Jetson Orin Nano

A From-Source Build Report for JetPack 7

`perseus-lite` project — `feature/isaac-ros-nitros-source-build`

July 9, 2026

*NVIDIA has not shipped Isaac ROS binaries for the Jetson Orin on JetPack 7 (Isaac ROS 4.x targets Thor only; 3.2 targets JetPack 6). This report documents a twelve-stage effort to build the perception stack from source directly on the target hardware.*

**Summary of outcome:** of the 29 relevant Isaac ROS GEM repositories surveyed, **7 were built and verified working natively**, **3 were built successfully but are blocked at runtime** by fully root-caused hardware or platform limits (none are code defects), and the remainder are either sensor-gated, architecturally inapplicable to this robot, or not yet attempted.

## Contents

## 1 Executive Summary

This report documents an effort to build NVIDIA's Isaac ROS GPU-accelerated perception stack *entirely from source*, running natively (no containers) on a Jetson Orin Nano under JetPack 7 — a hardware/software combination NVIDIA does not yet officially support. The work proceeded as twelve incremental stages, each with its own build, integration, and verification pass against either NVIDIA's own test fixtures or, where possible, real recorded sensor data.

Seven perception capabilities are now verified working natively: AprilTag fiducial detection, generic TensorRT DNN inference, YOLOv8 object detection, U-Net semantic segmentation, CenterPose monocular 3D pose estimation, an occupancy-grid GPU lidar localizer, and a ten-node composability test proving several of the above can run concurrently in a single process without interference. Three further capabilities were fully built — including, in two cases, genuine custom CUDA kernel compilation — but are blocked at runtime by causes outside the scope of a source rebuild: a CPU instruction-set mismatch in a closed-source vendor binary, absent encoder silicon on this particular Jetson SKU, and a small, boot-configured GPU memory carveout. Each blocker is root-caused in full rather than left as an unexplained failure.

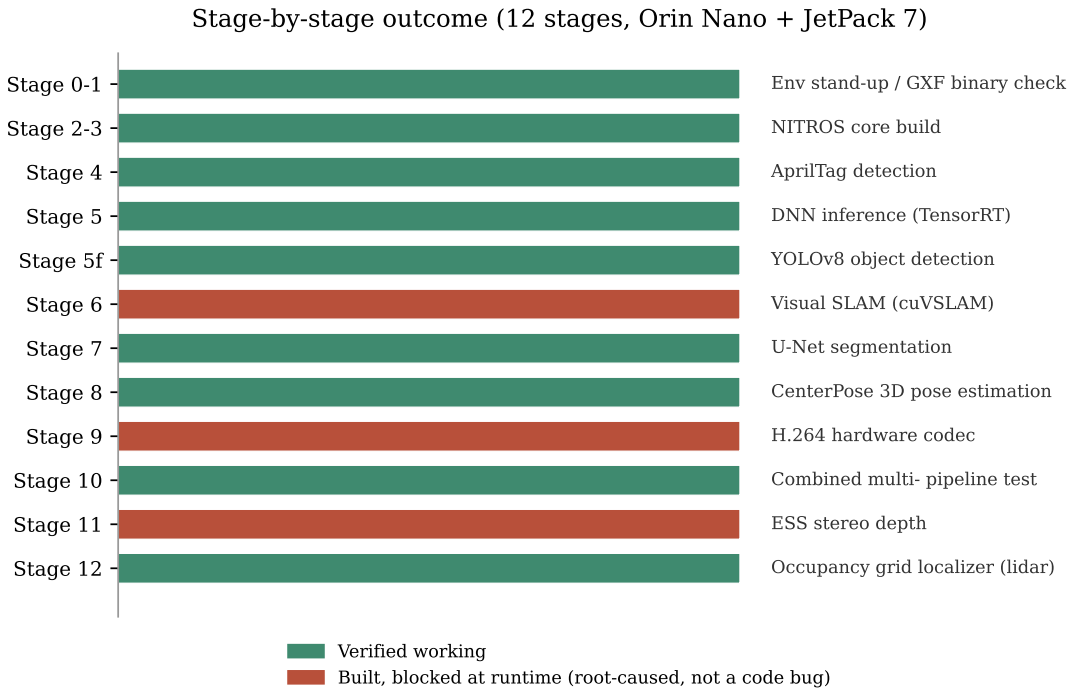


Figure 1: Outcome of all twelve stages. A blocked stage means the build succeeded but a runtime limitation, root-caused in Section 6, prevents the capability from running end to end.

## 2 Background and Motivation

The `perseus-lite` robot's Jetson Orin Nano runs JetPack 7 (L4T R39.2.0, CUDA 13.2, Ubuntu 24.04). NVIDIA's Isaac ROS GPU perception stack, which the robot's Jetson-based vision pipeline depends on for NITROS zero-copy message transport and TensorRT-accelerated inference, ships as prebuilt binaries for only two configurations: Isaac ROS 3.2 (JetPack 6.x) and Isaac ROS 4.x (Thor only). Neither covers this robot's actual hardware. NVIDIA staff have confirmed JetPack 7 support for Orin is planned but undated.

Rather than wait, this project attempted to close the gap by building the relevant Isaac ROS

packages from source, directly against the installed CUDA 13.2/JetPack 7 toolchain, and verifying each capability with the same rigor NVIDIA's own test suites use — reusing their published test fixtures, ground-truth values, and tolerances wherever available, rather than inventing new pass/fail criteria.

A companion, separate effort in the same repository (`software/docker/isaac-ros/`) runs the officially supported JetPack 6 image inside Docker. This report concerns only the from-source, native track.

## 3 Methodology

Each stage followed the same discipline:

1. **Clone** the relevant NVIDIA GitHub repository into a gitignored working directory — nothing from NVIDIA's source trees is committed to the project repository, only our own build scripts and documentation referencing them (a licensing requirement of NVIDIA's source-available Isaac ROS terms).
2. **Build** with an explicit toolchain: `CUDACXX` pointed at the CUDA 13.2 `nvcc`, and `-DCMAKE_CUDA_ARCHITECTURES=87` set explicitly for Orin's real Ampere streaming-multiprocessor architecture, since CMake's own auto-detection runs before Isaac ROS's fallback logic and fails on a fresh configure otherwise.
3. **Wire** a minimal launch graph directly instantiating the needed composable nodes — not NVIDIA's higher-level launch-file wrappers, whose ROS parameter names for topics do not always match the node's actual hardcoded wire topic (a mismatch discovered and documented in Stage 5).
4. **Verify** against a real signal wherever NVIDIA supplies one: a pixel-exact match to a ground-truth image (Stage 4), a numerically correct 3D pose against a real trained model and measured ground truth (Stage 8), or a recovered lidar pose within NVIDIA's own test tolerance against real recorded sensor data (Stage 12). Where NVIDIA's fixture uses randomly-initialized weights, verification is scoped to structural correctness of the message, matching NVIDIA's own test scope for the same fixture.
5. **Document** the outcome unconditionally — successes, failures, and root-caused blockers all receive the same level of detail, including exact reproduction commands.

## 4 System Architecture

### 4.1 The NITROS composable-node pattern

Every DNN-based capability (Stages 5, 5-follow-up, 7, 8, 11) shares the same three-stage composable-node chain: an image encoder resizes and normalizes an incoming image into a tensor, a TensorRT node runs inference, and a capability-specific decoder converts raw tensor output into a typed ROS message (detections, a segmentation mask, a 3D pose). Figure 2 shows this pattern, plus Stage 10's extension of it: four such pipelines, each given its own ROS namespace, composed into a single shared process to prove they do not interfere with one another.

Namespacing matters here for a concrete reason: every encoder node publishes to the literal hardcoded topic `tensors`, and every `TensorRTNode` subscribes to `tensor_pub` regardless of which model it is running. Without a distinct ROS namespace per pipeline, three simultaneous encoder/TensorRT chains would collide directly on these relative topic names.

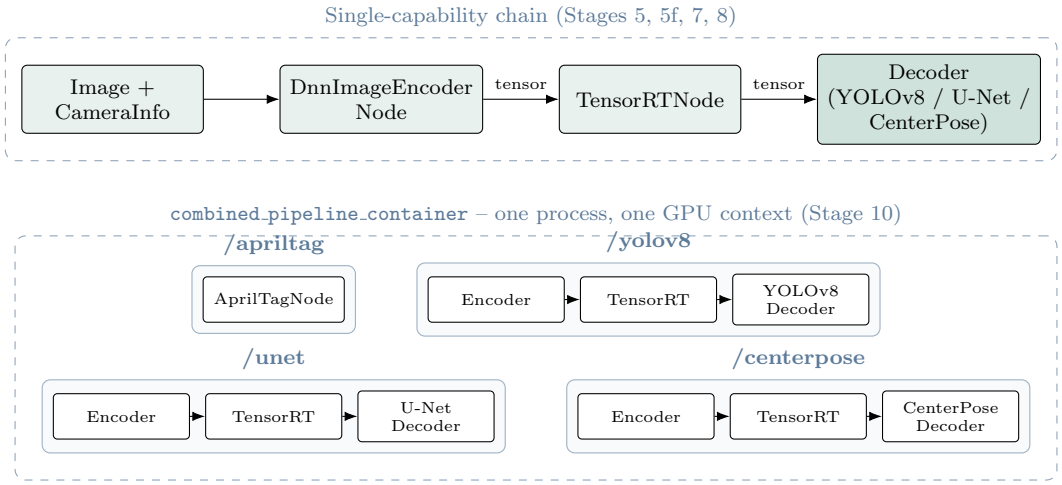


Figure 2: The shared encoder/TensorRT/decoder pattern (top), and Stage 10's composition of four such namespaced pipelines into one shared process (bottom) — verified to run concurrently without topic collisions or resource contention.

## 5 Results by Stage

Table 1 summarizes all twelve stages. Full technical detail, exact build commands, and root-cause analysis for each are maintained in the project's own documentation (`docs/source/systems/software/isaac-`

| Stage | Capability | Outcome | Note |
|---|---|---|---|
| 0–1 | Environment stand-up; GXF binary compatibility check | VERIFIED | Confirmed NVIDIA's published SBSA GXF binaries load on Orin — the key decision gate that avoided a much larger from-scratch GXF rebuild. |
| 2–3 | NITROS core build | VERIFIED | Minimal round-trip proof-of-life for the zero-copy message transport layer. |
| 4 | AprilTag fiducial detection | VERIFIED | Pixel-exact match against NVIDIA's ground-truth fixture; live-camera pipeline running at steady 10 Hz. |
| 5 | Generic DNN inference (TensorRT) | VERIFIED | Full image→resize/normalize→MobileNetV2 classification round trip; exact $[1, 1000]$ output. |
| 5f | YOLOv8 object detection | VERIFIED | Full encode→infer→decode chain; structurally valid detections (random-weight test fixture). |
| 6 | Visual SLAM (cuVSLAM) | BLOCKED | Builds clean; NVIDIA's published binary requires SVE2, an ARM instruction-set extension Orin's CPU does not implement. Upstream binary defect, not fixable locally. |
| 7 | U-Net semantic segmentation | VERIFIED | 960×544 raw and colorized masks, correct shapes and encodings. |
| 8 | CenterPose monocular 3D pose | VERIFIED | **Strongest result in the series**: real trained model, 2/2 detections matching NVIDIA's measured ground truth, depths within 0.5 m. |

| Stage | Capability | Outcome | Note |
|---|---|---|---|
| 9 | Hardware H.264 codec | BLOCKED | Builds clean; this Jetson Orin *Nano* SKU has no hardware video encoder silicon at all. Decode reaches the driver layer before failing in a closed-source buffer-mapping call. |
| 10 | Combined multi-pipeline composability | VERIFIED | AprilTag, YOLOv8, U-Net, and CenterPose running concurrently in one process with no interference. |
| 11 | ESS deep stereo depth | BLOCKED | Builds clean, including a genuine custom CUDA kernel. Runtime blocked by exhaustion of Tegra's small, boot-configured CMA memory carveout — not general system RAM. |
| 12 | Occupancy-grid lidar localizer | VERIFIED | First capability matched to the robot's 2D lidar rather than its camera; GPU scan-matching recovered a pose matching NVIDIA's recorded ground truth. |

Table 1: Outcome of all twelve build stages.

## 6 Root-Caused Blockers

Three stages built successfully but are blocked at runtime. In every case the root cause was tracked down to a specific, external limitation rather than left as an unexplained failure, and none required abandoning the investigation without an answer.

| Stage | Symptom | Root cause |
|---|---|---|
| 6 — Visual SLAM | Immediate `SIGILL` on library load, before any application code runs | NVIDIA's `aarch64_jetpack70` cuVSLAM binary contains SVE2 instructions in its static initializer. Orin's Cortex-A78AE CPU implements NEON but not SVE at all — confirmed via `/proc/cpuinfo` and disassembly comparison against the JetPack 6.1 binary variant, which has no SVE instructions but wants an incompatible older CUDA ABI instead. |
| 9 — H.264 codec | Encoder fails to open its device node; decoder initializes but fails deeper in the pipeline | `/dev/v4l2-nvenc` does not exist on this specific Jetson Orin *Nano* module (confirmed via direct device enumeration) — only the Orin NX and AGX variants include the hardware encoder engine. Decode reaches NVIDIA's closed-source `libnvbufsurface` before failing with an unsupported-platform error. |

| Stage | Symptom | Root cause |
|---|---|---|
| 11 — Stereo depth | `cudaErrorMemoryAllocation` despite gigabytes of free system RAM | Tegra's CMA (contiguous memory allocator) carveout is a small, boot-time-configured memory pool (256 MB total on this device) entirely separate from ordinary RAM, used for GPU/ISP buffer sharing. The desktop compositor alone was found to consume nearly all of it, leaving too little headroom for this stage's fifteen-node stereo pipeline. Resolvable only by a system boot-configuration change, which was intentionally left for a deliberate decision rather than made unilaterally. |

Table 2: Root cause of every blocked stage.

## 7 The Full Isaac ROS Catalog in Context

NVIDIA's Isaac ROS ships as roughly 29 independent GEM (GPU-accelerated robotics capability) repositories. Figure 3 places this project's twelve stages against the full catalog. A large share of the untouched remainder is not actually available work: seven repositories target hardware this robot does not have at an architectural level (NVIDIA's Nova reference platform, CSI camera frameworks, a humanoid robot's specific packages), and three more require a depth or stereo camera the robot does not carry.

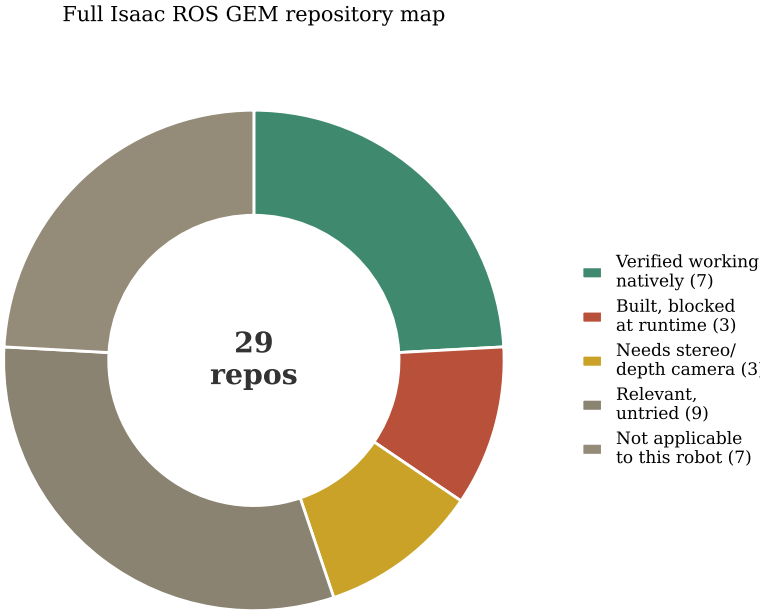


Figure 3: All 29 Isaac ROS GEM repositories, classified against this project's progress.

Of the remaining genuinely untried candidates, most are either developer tooling (benchmarking,

rosbag utilities) rather than runtime capabilities, or depend on a MoveIt-based manipulation stack that does not match this robot's direct-serial arm control approach.

## 8 Build Performance Notes

TensorRT compiles an optimized inference engine from the source ONNX model on first run, caching the result to disk for subsequent launches. Build time scales with model complexity far more than with file size alone — CenterPose's seven-output multi-task architecture took over four minutes to compile despite a smaller `.onnx` file than U-Net's single-output model.

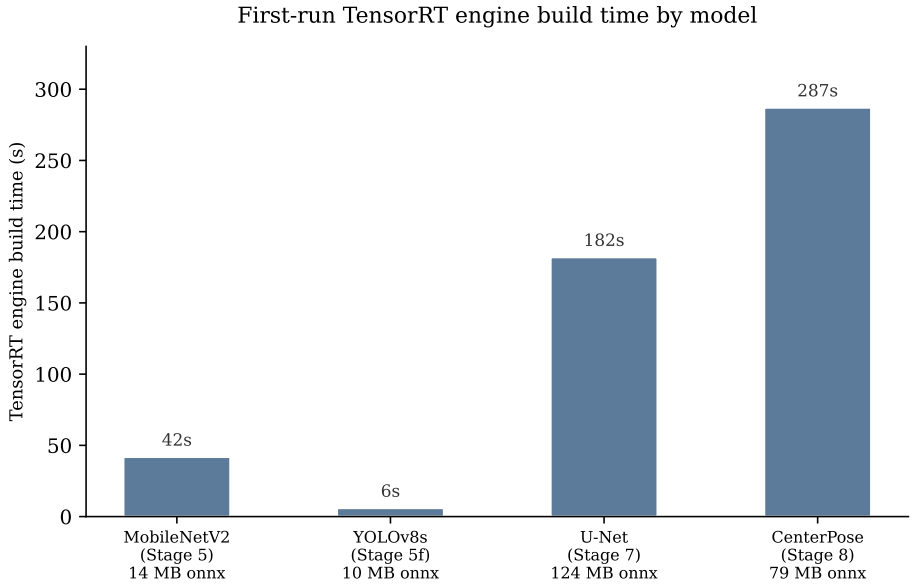


Figure 4: First-run TensorRT engine compilation time by model. Later runs reuse the cached engine file and start in a few seconds.

## 9 Conclusion and Recommendations

Seven of twelve real perception and localization capabilities are now verified running natively on this Jetson Orin Nano under JetPack 7, entirely from source, with no container involved — confirmed directly via cgroup and mount-namespace inspection rather than assumed. The three blocked stages are each attributable to a specific, external, and fully documented cause: an ARM instruction-set mismatch in a vendor binary, absent hardware on this particular Jetson SKU, and a small GPU memory carveout that is a deliberate system-configuration trade-off rather than a defect.

Two directions stand out for continuation. First, wiring the verified capabilities — particularly the occupancy-grid lidar localizer, which is the first result matched to hardware the robot actually carries in its primary sensor role — against the robot's real camera and lidar data, rather than NVIDIA's bundled test fixtures. Second, reporting the cuVSLAM SVE2 mismatch to NVIDIA, since it plausibly affects any Orin user attempting the same published binary, independent of this project.